1ª AVALIAÇÃO · ENGENHARIA DE DADOS

OlistFlow

Protótipo de Ciclo de Vida de Engenharia de Dados

Planejamento e Desenho Arquitetural · Parte 1 de 2

RL APRESENTADOR
Rodrigo César Lins

VF APRESENTADOR
Vitor Nascimento Franco

30/04/2026 · 15 min + Q&A · Parte 1 de 2

Por que um marketplace brasileiro?

R$ 185 bi

E-COMMERCE BRASIL · 2024
Marketplaces concentram a maior complexidade de dados

Cada pedido envolve **cliente, vendedor, meio de pagamento e operador logístico** — em todos os estados, com ciclos longos da compra à avaliação. Os dados nascem dispersos em sistemas que não conversam.

Três dores recorrentes

- **Visão fragmentada** — cada equipe consulta uma fonte diferente
- **Decisões lentas** — "qual categoria cresceu?" leva dias
- **Sinais perdidos** — navegação nunca cruza com pós-compra

> *O OlistFlow ataca essas três dores com um pipeline end-to-end unificando dados operacionais (batch) e comportamentais (streaming).*

Dois tipos de dados, duas histórias

Dimensão	📦 Batch (Operacional)	⚡ Streaming (Comportamento)
Fonte	PostgreSQL simulando ERP Dataset Olist — 9 tabelas reais	Simulador Python → Redis Streams Fallback: JSONL em disco
Formato	Tabelas relacionais → Parquet	Eventos JSON → Parquet
Volume	~1,5 M linhas · 45 MB	~50 k eventos/dia · 5 MB/dia
Frequência	Diária (D-1, noturno)	Contínua, micro-batch 5 min
Latência SLA	Dashboard até 6h depois	≤ 5 minutos
Exemplos	Pedidos, pagamentos, avaliações, cadastros	page_view, search, add_to_cart, checkout_abandon

Tratamos os dois porque contam histórias diferentes da mesma jornada.

Domínios — quem quer o quê

💰 Vendas
Liderança · Analistas
GMV, ticket médio, conversão, mix de categoria

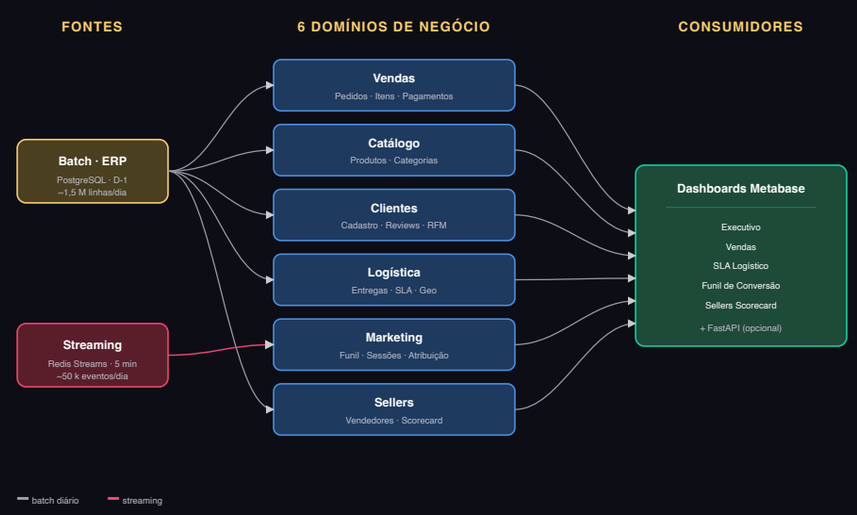
📦 Catálogo
Todos os domínios
Produtos e categorias canônicas — base de referência

👤 Clientes
Marketing · Vendas
Segmentação RFM, perfil do comprador, reviews

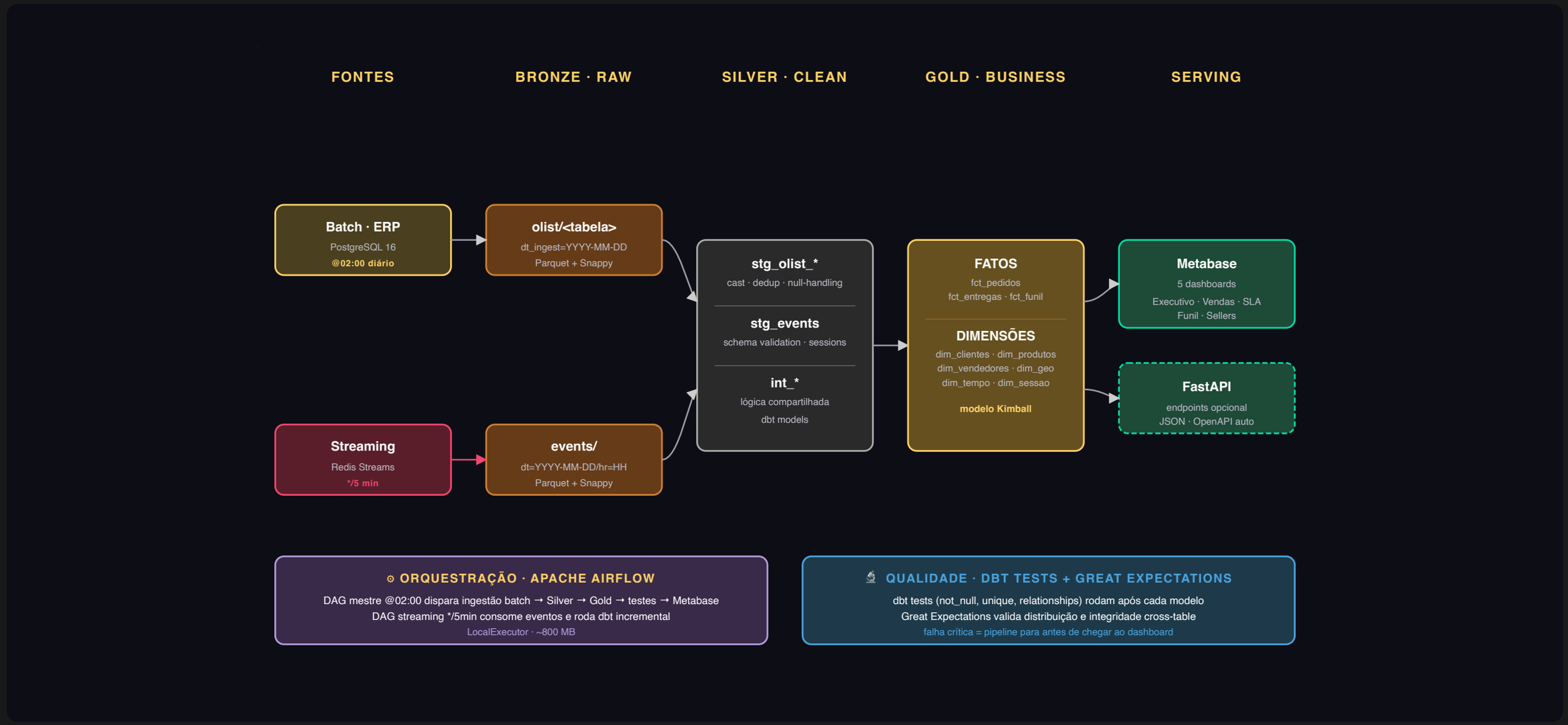
🚚 Logística
Operações
SLA de entrega, gargalos por rota, atrasos

📣 Marketing
Produto · Marketing
Funil de conversão, abandono de carrinho, atribuição

🏪 Sellers
Gestão de marketplace
Scorecard de vendedor, NPS, taxa de cancelamento

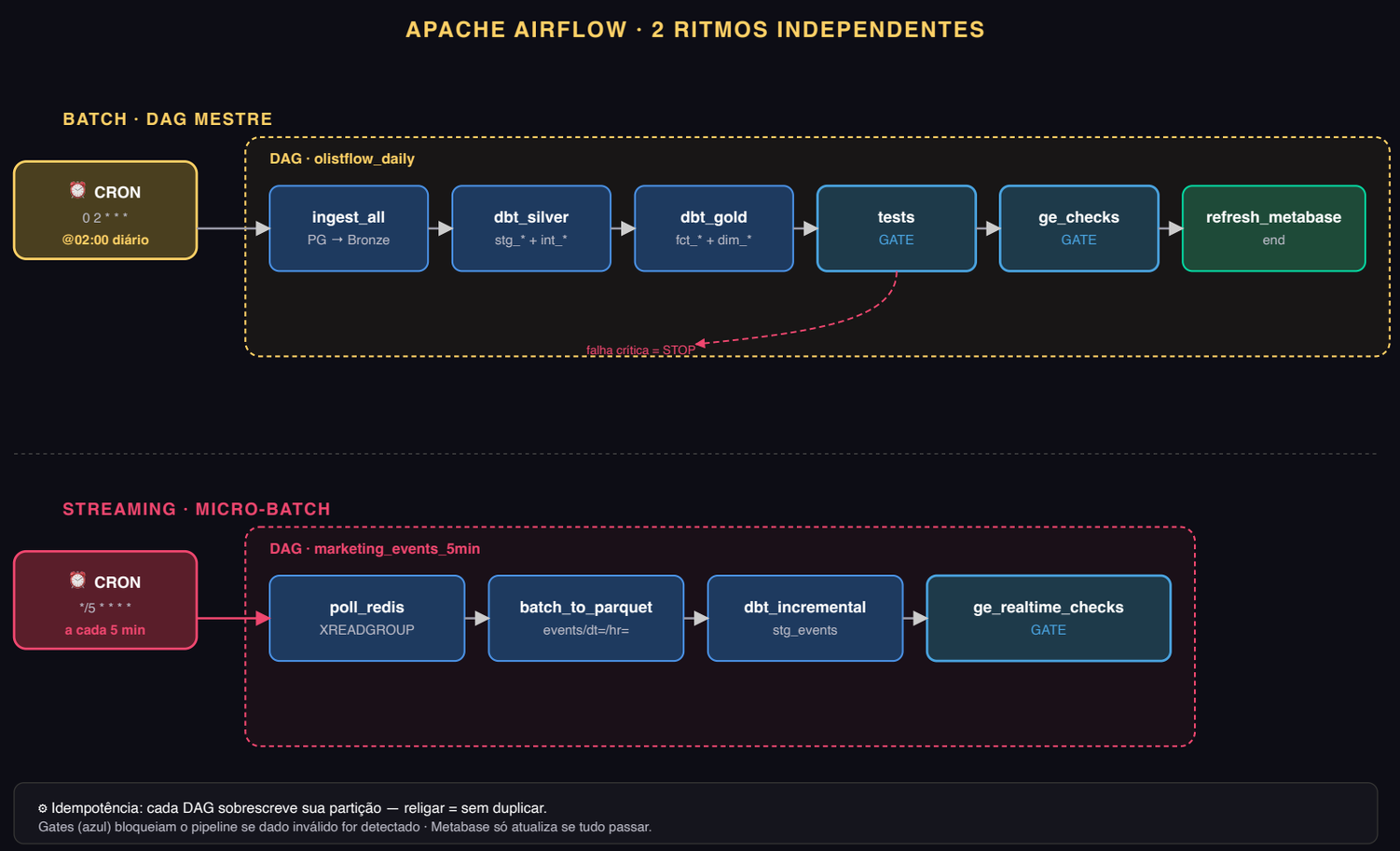


Arquitetura Medalhão + Lambda light



Bronze captura · **Silver** limpa · **Gold** modela · cadência anotada no próprio fluxo

Como Airflow coordena — DAGs e schedules



Dois ritmos independentes: **trem diário @02:00** (sequencial) + **streaming */5min** (fire-and-forget).

Por que Medalhão e não outras?

Arquitetura	Por que não ganhou
Data Warehouse puro	Carrega dados já transformados — perde raw, não reprocessa. Inadequado para eventos semi-estruturados.
Data Lake puro	Sem estrutura vira "pântano de dados". Não cobre confiabilidade exigida.
Lambda puro (Kafka + Spark)	Overkill. Duas stacks completas exigem hardware inviável em contexto acadêmico.
Kappa puro	Trata tudo como streaming. Desproporcional — maior parte dos dados é batch por natureza.
Data Mesh	Exige múltiplos times com ownership. Uma dupla de estudantes não reproduz isso.
Medalhão ✅	**Vencedor:** cobre requisitos, roda em notebook, expressa princípios de camadas/acoplamento/reversibilidade.

🪶 Cabe no hardware

DuckDB in-process · Parquet local · sem cluster

⏪ Reversibilidade

Bronze imutável · qualquer bug reconstrói tudo

🚀 Migrável

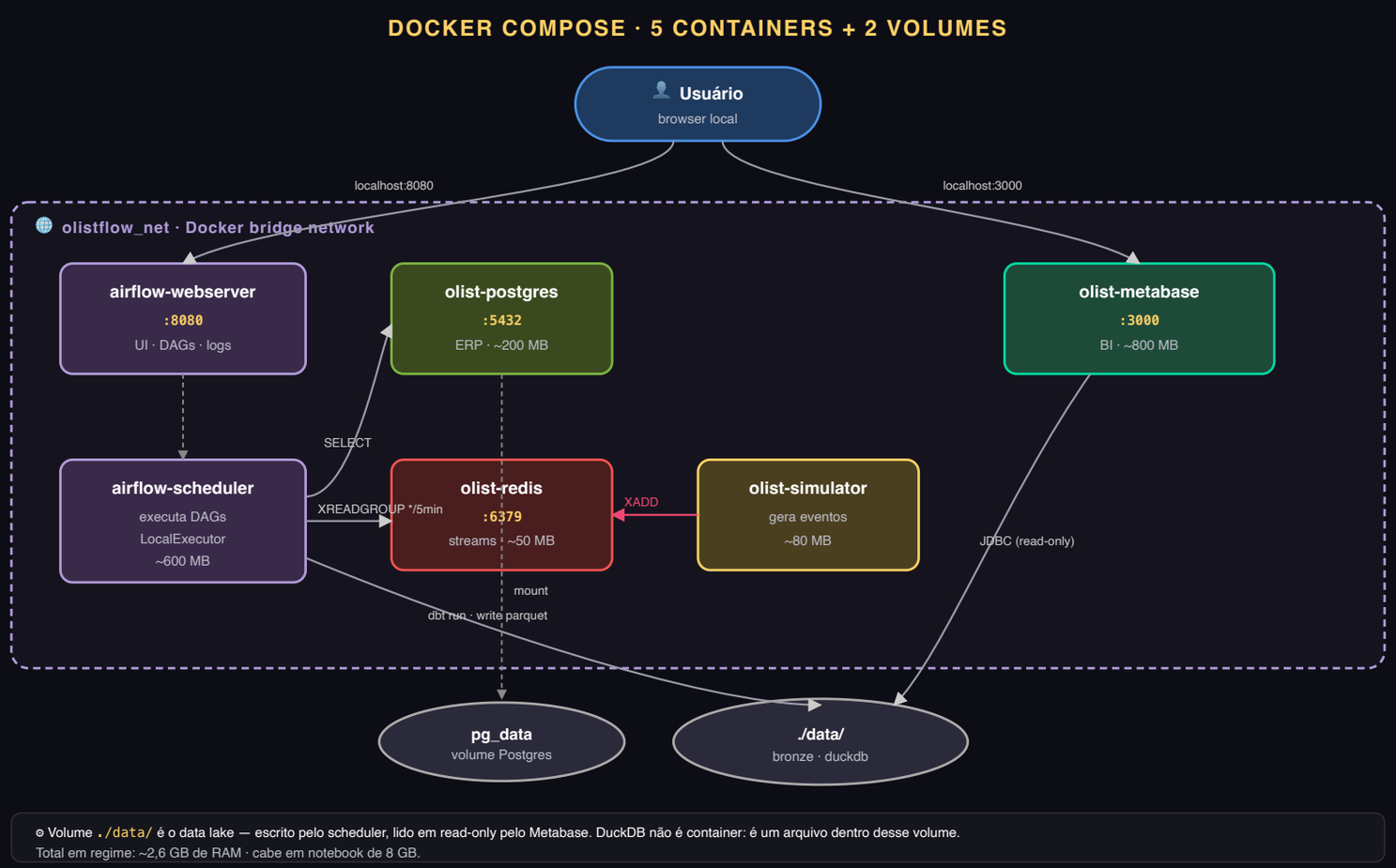
Troca DuckDB→BigQuery e Parquet→S3 sem redesenho

Stack Tecnológico

Etapa	Ferramenta	Alternativa futura (cloud)	RAM
Ingestão batch	**Python + pyarrow**	Airbyte, Fivetran, Debezium CDC	100 MB
Ingestão streaming	**Redis Streams**	Kafka MSK, Kinesis, Pub/Sub	150 MB
OLTP origem	**PostgreSQL 16**	RDS, Cloud SQL	200 MB
Data Lake	**Parquet + diretórios**	S3, GCS, Azure Blob	0
Engine analítica	**DuckDB**	BigQuery, Snowflake, Athena	300 MB
Transformação	**dbt-core**	dbt Cloud	50 MB
Orquestração	**Apache Airflow**	MWAA, Cloud Composer	800 MB
BI / Dashboards	**Metabase**	Looker, Power BI	800 MB
Qualidade	**dbt tests + Great Expectations**	Soda, Monte Carlo	100 MB
		Total estimado em RAM:	**~2,6 GB**

Cabe em notebook de 8 GB com folga para sistema + IDE + navegador. Toda a stack é **open-source** — zero custo de licença.

Como roda na máquina — Docker Compose



5 containers na mesma bridge + volumes locais para código (hot-reload) e dados (persistência). **~2,6 GB em regime**, cabe em notebook de 8 GB.

Três decisões não-triviais

🦆 DuckDB sobre Spark

Pra 45 MB de dados, Spark seria overkill de ordem de grandeza.

DuckDB in-process. SQL ANSI. Integração direta com dbt.

⚡ Redis Streams sobre Kafka

Kafka exige ZooKeeper + brokers + 2 GB só em overhead.

Redis tem a mesma semântica de log imutável em 50 MB.

🛠️ dbt como espinha

Transforma SQL em código versionável com testes e linhagem automática.

É o padrão de mercado hoje — vale aprender.

> *Cada escolha tem **plano B documentado** no relatório. Se Redis não rodar na sala, o simulador grava direto em JSONL — mesmo pipeline, menos infra.*

Tradeoffs honestos

✅ O que ganhamos

- **Baixo acoplamento:** camadas conversam via Parquet com schema contratado
- **Confiabilidade:** dbt tests + GE bloqueiam dados inválidos
- **Reversibilidade:** Bronze imutável permite `--full-refresh`
- **Linhagem automática:** dbt gera grafo completo
- **Portabilidade:** mesma topologia roda em S3/BigQuery

⚠ O que cedemos conscientemente

- **Alta disponibilidade real:** single-node, falha = downtime
- **Streaming industrial:** Redis < Kafka; volumes didáticos
- **RBAC granular no serving:** Metabase com auth simples
- **Data catalog corporativo:** usamos dbt docs local
- **Escala horizontal:** DuckDB é single-node

*Todas as renúncias são **recuperáveis** em uma versão futura — sem alterar a topologia principal.*

Parte 2 — 5 ondas de implementação

1
Fundação
docker-compose sobe stack · Postgres recebe Olist · dbt inicializado

2
Bronze
DAG ingest_olist · simulador de eventos · Parquet particionado

3
Silver
stg_* + int_* · cast, dedup, validação · dbt tests

4
Gold + QA
dim_* + fct_* · snapshots SCD2 · suite Great Expectations

5
Consumo
5 dashboards Metabase · dbt docs · CI GitHub Actions

Critério de pronto

- ✅ `make up` sobe todo o stack em < 5 min em máquina limpa
- ✅ `dbt build` roda do zero e passa em 100% dos testes
- ✅ Qualquer colega da sala consegue reproduzir em seu notebook em < 30 min

ENCERRAMENTO

Obrigado!

OlistFlow — Parte 1 entregue.

RL APRESENTADOR
Rodrigo César Lins

VF APRESENTADOR
Vitor Nascimento Franco

📂 Repositório público

`github.com/rodrigolinss/olistflow-dataeng-proto`

Slides em `/apresentacao/` · Relatório em `/docs/`